

4G Microservices Cloud Enablement Platform

Complete Technical & Interview Reference — DevOps/DevSecOps Engineer (4 YOE)

Telstra Platform (via HCL Technologies) — Project Deep Dive

0. How to Open This in an Interview
1. Full AWS Infrastructure Architecture
 - 1.1 Region & AZ Layout
 - 1.2 VPC & CIDR Design
 - 1.3 EKS Control Plane — What AWS Manages vs. What You Manage
 - 1.4 Worker Node Distribution (76+ nodes)
 - 1.5 EKS Cluster Upgrade Process
2. Docker / Container Image Specifics
3. Terraform — Infrastructure as Code
4. Ansible — Configuration Management
5. Jenkins — Full End-to-End Pipeline
6. Kubernetes / GitOps Folder Structure
7. Database, Storage & Backup — RDS, S3, EBS
8. Disaster Recovery Strategy
9. Monitoring & Health Checks
10. Secrets Management — AWS Secrets Manager
11. IAM Roles & Policies (be ready to describe these specifically)
12. Kubernetes Security — IRSA & RBAC in Detail
 - 12.1 IRSA (IAM Roles for Service Accounts)
 - 12.2 RBAC (Role-Based Access Control) — the Kubernetes-native layer, separate from IRSA
13. The Biggest Problem Faced — and How It Was Solved
14. Biggest Contribution to the Project
15. Defending the 3 Key CV Metrics — In Detail
 - Metric 1: 40% Reduction in Deployment Time
 - Metric 2: 25% Improvement in Resource Utilization
 - Metric 3: 99.99% Uptime for Critical Production Systems
16. Comprehensive Interview Q&A Bank (Additional to Earlier Sections)
17. Approval Gates — Summary Across Every Tool
18. Numbers to Defend, One More Time
19. The One Rule for the Actual Interview
21. Visual: AWS Infrastructure Architecture
22. Visual: Traffic Flow — External World to Pod (with Service Mesh)
23. Visual: CI/CD + Multi-Cluster Overview (the whole pipeline, one picture)
24. How Pods Communicate With Each Other
25. Deployment Strategies — Rolling vs. Canary vs. Blue-Green (when each is used)
26. Agile Methodology — How This Was Delivered
27. Detailed Folder Structures (Full Repos)
 - 27.1 Terraform Repo
 - 27.2 Ansible Repo
 - 27.3 Docker / Application Repo (per microservice)
 - 27.4 Jenkins (shared library repo)
 - 27.5 GitOps / Kubernetes Manifests Repo
28. Git Repository & Branching Strategy
29. Terraform & Ansible Pipeline Execution
 - Terraform pipeline (as a Jenkins job, separate from the app pipeline)
 - Ansible pipeline
 - How Terraform state is maintained across environments
30. Jenkins Pipeline Execution Across All Environments

- 31. Alerting & Monitoring — What We Actually Scrape and Alert On
 - Metrics scraped by Prometheus
 - Alerting rules (examples worth naming specifically)
 - Dashboards (Grafana)
- 32. Kubernetes Ingress, Services, and Stateful vs. Stateless Pods
 - Ingress
 - Services
 - Stateless vs. Stateful — how the split was actually made
- 33. DaemonSet Pods Run in the Cluster
- 34. End-to-End Project Component & Tool Map (the one-page summary)

0. How to Open This in an Interview

"I worked as a DevOps engineer on Telstra's 4G microservices cloud platform — migrating legacy telecom-supporting applications into containerized microservices on multi-AZ AWS EKS. I owned the CI/CD pipeline build-out in Jenkins with integrated security gates, managed our Kubernetes workloads and Helm/GitOps delivery via ArgoCD, and ran monitoring/alerting with Prometheus and Grafana. The platform ran 76+ worker nodes across multiple Availability Zones in a single region, with a warm-standby DR posture in a secondary region, and sustained 99.99% uptime. I worked closely with our cloud architects on the underlying Terraform infrastructure and contributed to cost optimization by right-sizing based on real utilization data."

Ownership calibration (use throughout): you own CI/CD, Kubernetes operations, Helm/GitOps for your services, and monitoring/alerting for your services. You *contributed to* — not solely designed — the VPC architecture, org-wide IAM strategy, and DR policy decisions.

1. Full AWS Infrastructure Architecture

1.1 Region & AZ Layout

- Single primary AWS region (e.g. `ap-southeast-2`) — chosen for data residency, since telecom subscriber data typically must stay in-country.
- **3 Availability Zones** in the production VPC — this is the “multi-zone” architecture from the CV.
- Secondary DR region kept in-country where residency requires it, run as pilot-light/warm-standby only (see Section 8).

1.2 VPC & CIDR Design

VPC CIDR: 10.0.0.0/16 (65,536 IPs – sized generously because the AWS VPC CNI gives every pod a real routable IP, not an overlay IP, so address space depletes faster than node count alone suggests)

AZ-a

Public

10.0.0.0/24

Private (EKS nodes)

10.0.16.0/20

AZ-b

Public

10.0.1.0/24

Private (EKS nodes)

10.0.32.0/20

AZ-c

Public

10.0.2.0/24

Private (EKS nodes)

10.0.48.0/20

Reserved for growth / DB subnet group: 10.0.64.0/20, 10.0.80.0/20, 10.0.96.0/20

- **NAT Gateway per AZ** (not shared) — avoids cross-AZ data transfer charges and removes a single point of failure for outbound traffic.
- **Route tables:** public subnets → Internet Gateway; private subnets → NAT Gateway in the same AZ.
- **Security Groups** scoped per tier: ALB SG (443 from internet only), Node SG (traffic from ALB SG + intra-cluster), RDS SG (DB port from Node SG only) — nothing broad-open.
- **VPC Flow Logs** enabled to CloudWatch/S3 — feeds the audit/compliance logging pipeline.

1.3 EKS Control Plane — What AWS Manages vs. What You Manage

Component	Ownership
etcd (cluster state store)	Fully AWS-managed. No customer access, no customer backup responsibility.
API server, scheduler, controller-manager	Fully AWS-managed , multi-AZ by default, patched by AWS
Control plane version upgrade	You initiate , AWS executes
Worker nodes (EC2)	You fully own — AMI, patching, scaling, node groups
Add-ons (VPC CNI, CoreDNS, kube-proxy)	You own version and upgrade timing

If asked “how do you back up etcd”: “On EKS, etcd is fully AWS-managed — we have no access to it and no responsibility to back it up; that’s covered under the EKS SLA. What we own instead is backing up cluster state as we define it — Terraform for infrastructure, and Velero for Kubernetes objects and persistent volume snapshots — so we can reconstruct a cluster’s actual workloads even though we never touch etcd directly.” This is the technically correct answer and signals real understanding rather than a memorized buzzword.

1.4 Worker Node Distribution (76+ nodes)

Node Group 1 – "critical-ondemand"

m5.xlarge (4 vCPU/16GB) | On-Demand | ~30 nodes (10/AZ)

Workload: subscriber-facing core services, stateful services

Taint: workload-type=critical:NoSchedule

Node Group 2 – "general-spot"

m5.xlarge / m5a.xlarge / m5.2xlarge (diversified) | Spot | ~40 nodes (~13/AZ)

Workload: stateless internal services, batch/reporting jobs

Node Group 3 – "platform-addons"

m5.large | On-Demand | ~6 nodes (2/AZ)

Workload: ArgoCD, Prometheus/Grafana, ingress-nginx, External Secrets Operator, Velero

(kept separate so app workloads never starve platform tooling of resources)

- **Karpenter** provisions additional capacity within seconds of unschedulable pods and consolidates underutilized nodes automatically — the modern replacement for Cluster Autoscaler.

- **Taints/tolerations** enforce workload separation; **resource requests/limits are mandatory** on every Deployment, enforced by a Kyverno admission policy.

1.5 EKS Cluster Upgrade Process

EKS upgrades move **one minor version at a time** (e.g., 1.28 → 1.29) — an AWS-enforced constraint worth knowing explicitly.

1. Run `kubent / pluto` to detect deprecated API versions in current manifests.
 2. Upgrade control plane via Terraform (`cluster_version` bump + apply) — no worker impact, AWS handles it.
 3. Upgrade EKS add-ons (VPC CNI, CoreDNS, kube-proxy) to compatible versions.
 4. Upgrade node groups via **blue/green replacement**, never in-place:
 - Create new node group with new AMI
 - `kubectrl cordon` old nodes → `kubectrl drain --ignore-daemonsets --delete-emptydir-data`
 - Validate workload health on new nodes
 - Delete old node group once fully drained and confirmed healthy
 - (Karpenter automates most of this via node “drift” detection)
 5. Promotion order: dev → staging → prod, each validated, prod done in a scheduled maintenance window with the old launch template kept available for fast rollback.
-

2. Docker / Container Image Specifics

Base images (specific, defensible choices, not “whatever’s default”): - Java: multi-stage — `maven:3.9-eclipse-temurin-17` (build) → `gcr.io/distroless/java17-debian12` or `eclipse-temurin:17-jre-alpine` (runtime) - Node.js: `node:20-alpine` (build) → `gcr.io/distroless/nodejs20-debian12` - Python: `python:3.11-slim` for both stages, build tools dropped from the final layer

```
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM gcr.io/distroless/java17-debian12
WORKDIR /app
COPY --from=builder /app/target/subscriber-service.jar app.jar
USER 1001
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Non-root `USER` is set in the Dockerfile **and** re-enforced by a Kyverno `runAsNonRoot: true` admission policy — defense in depth even if a Dockerfile forgets it. Image tags are immutable git-commit-SHA, never `latest` past dev.

3. Terraform — Infrastructure as Code

```
terraform/  
├─ modules/{vpc, eks, iam, rds, ecr}/  
├─ environments/{dev, staging, prod, dr}/    (own backend.tf + tfvars each)  
└─ global/state-backend/                  (S3 + DynamoDB bootstrap, applied once)
```

- Remote state: S3 (versioned, encrypted) + DynamoDB lock table; **separate state file per environment**, not workspaces, to prevent cross-env accidents.
 - `tfsec` / `checkov` run on every PR touching Terraform — blocking gate on misconfigurations (open SGs, unencrypted resources, public buckets).
 - Region is a locked variable per environment, enforced via a `variables.tf` validation block — this is the data-residency control, implemented in code.
 - **Approval gate:** `plan` posted to PR for review → 1-2 approvers required → auto-apply for dev/staging → **manual approval step for prod apply**, run by a senior engineer in an agreed change window.
-

4. Ansible — Configuration Management

```
ansible/
├── inventories/{dev,staging,prod}/hosts.ini
├── group_vars/
├── roles/{cis-hardening, monitoring-agent, bastion-setup}/
└── playbooks/site.yml
```

- **Ansible Vault** encrypts sensitive variables at rest; vault password injected at runtime via a Jenkins credential.
 - Every playbook run with `--check --diff` (dry run) in CI before the real apply.
 - Idempotency enforced via native Ansible modules rather than raw `shell` / `command` tasks.
-

5. Jenkins — Full End-to-End Pipeline

Checkout -> Unit Tests -> SonarQube SAST -> Quality Gate (BLOCK) ->
Dependency/SCA Scan (BLOCK) -> Build Docker Image ->
Trivy Image Scan (BLOCK on HIGH/CRITICAL) -> Cosign Sign Image ->
Push to ECR -> Helm Lint + kubeconform Validate -> Helm Dry-Run ->
Update GitOps Repo (image tag bump) -> [ArgoCD auto-detects & syncs] ->
Prod Approval Gate -> Post-Deploy Smoke Test -> Slack Notification

```
@Library('devops-shared-library') _

pipeline {
    agent {
        kubernetes {
            label 'jenkins-agent'
            defaultContainer 'builder'
            yamlFile 'ci/jenkins-agent-pod.yaml'
        }
    }
    environment {
        AWS_ACCOUNT_ID = credentials('aws-account-id')
        ECR_REPO        = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-southeast-2.amazonaws.com/subscriber-service"
        IMAGE_TAG       = "${env.GIT_COMMIT.take(7)}"
    }
    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Unit Tests') { steps { sh 'mvn test' } }
        stage('SonarQube Analysis') {
            steps { withSonarQubeEnv('sonar-server') { sh 'mvn sonar:sonar -Dsonar.projectKey=subscriber-service' } }
        }
        stage('Quality Gate') {
            steps { timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true } }
        }
        stage('Dependency Scan') { steps { sh 'dependency-check.sh --project subscriber-service --failOnCVSS 7 --sc' } }

        stage('Build Image') { steps { sh 'docker build -t ${ECR_REPO}:${IMAGE_TAG} .' } }
        stage('Image Vulnerability Scan') { steps { sh "trivy image --severity HIGH,CRITICAL --exit-code 1" } }
        stage('Sign Image') { steps { sh "cosign sign --key awskms:///alias/cosign-signing-key ${ECR_REPO}:${IMAGE_TAG}" } }

        stage('Push to ECR') {
            steps { sh """
aws ecr get-login-password --region ap-southeast-2 | docker login --username AWS --password-stdin
${ECR_REPO}

docker push ${ECR_REPO}:${IMAGE_TAG}
""" }
        }
        stage('Helm Lint & Validate') {
            steps {
                sh 'helm lint ./charts/subscriber-service --values ./charts/subscriber-service/values-prod.yaml'
                sh 'helm template ./charts/subscriber-service --values values-prod.yaml | kubeconform -strict -summ'
            }
        }
        stage('Update GitOps Manifest') {
            steps { sshagent(['gitops-repo-ssh-key']) { sh """
git clone git@github.com:telstra-platform/gitops-manifests.git
cd gitops-manifests/apps/subscriber-service/overlays/prod
yq -i '.image.tag = "${IMAGE_TAG}"' values.yaml
git commit -am "ci: bump subscriber-service to ${IMAGE_TAG}"
git push origin main
""" } }
        }
        stage('Prod Approval Gate') {
            when { branch 'main' }
            steps { timeout(time: 30, unit: 'MINUTES') { input message: 'Approve production sync?', submitter: 'dev' } }
        }
    }
    post {
        success { slackSend(color: 'good', message: "Deployed ${IMAGE_TAG}") }
        failure { slackSend(color: 'danger', message: "Pipeline failed") }
    }
}
```

6. Kubernetes / GitOps Folder Structure

```
gitops-manifests/
├── argocd/
│   ├── root-app.yaml          # App-of-Apps entrypoint
│   └── applicationsets/       # multi-cluster templating
├── apps/<service-name>/
│   ├── base/                  # shared Deployment, Service, HPA, ServiceMonitor
│   └── overlays/{dev,staging,prod,dr}/values.yaml
└── infra-apps/
    ├── prometheus-stack/
    ├── external-secrets-operator/
    ├── ingress-nginx/
    ├── kyverno/
    └── velero/
```

base/ defines *what* a service looks like; **overlays/** define *how it differs per environment*. ArgoCD sync policy: dev/staging = automated + self-heal + prune; **prod = manual sync only**, triggered after the Jenkins approval gate passes — two independent approval layers before anything touches production.

7. Database, Storage & Backup — RDS, S3, EBS

Microservice pod (EKS)

- > Transactional data -> RDS (PostgreSQL/Aurora), one DB per service
 - Multi-AZ (sync standby, ~60-120s auto-failover)
 - Automated daily snapshots, 7-35 day PITR retention
 - Cross-region read replica (DR promotion target)
 - RDS Proxy for connection pooling (prevents pod-churn exhausting DB connections)
- > Block storage -> EBS (gp3) via EBS CSI Driver
 - Dynamic PVC provisioning per StatefulSet
 - Snapshoted via AWS Backup (scheduled) + captured in Velero backups
- > Object storage -> S3
 - Versioning enabled, SSE-KMS encryption at rest
 - Lifecycle: Standard -> IA (30d) -> Glacier (90d) -> expire per policy
 - Cross-Region Replication to DR bucket (Velero backups, TF state, artifacts)

Velero backs up Kubernetes object definitions + EBS snapshots (scheduled daily + on-demand before risky changes). **RDS** backups cover the transactional data with its own point-in-time-recovery semantics — a different system with a different retention policy, by design. Together, a full environment (cluster config + data) can be reconstructed in the DR region without manual rebuild.

8. Disaster Recovery Strategy

Primary Region	DR Region (in-country if residency requires)
EKS full scale (76 nodes)	Velero+S3 CRR EKS pilot-light (minimal/scaled-to-0)
RDS primary, Multi-AZ	-- replication --> RDS cross-region read replica
ECR	-- replication --> ECR replica
Route53 health-check failover routing ties the two together	

Runbook: detect (Route53 health checks / Prometheus blackbox) -> decision gate (automated for confirmed region outage, manual for ambiguous cases) -> promote RDS replica -> scale up DR node groups (Karpenter reacts in seconds) -> ArgoCD syncs manifests into the now-active DR cluster -> Route53 DNS cutover -> validate via smoke tests/Grafana -> failback during a maintenance window once primary is restored.

Honest RTO/RPO: pilot-light gives an RTO in the tens-of-minutes range and RPO bounded by replication lag (minutes) — not near-zero. State that trade-off plainly if asked; it shows you understand *why* you didn't build active-active (cost/complexity vs. requirement).

9. Monitoring & Health Checks

Layer	Mechanism
Pod	<code>livenessProbe</code> (restart if unhealthy) + <code>readinessProbe</code> (remove from Service endpoints)
Service/Ingress	ALB target group health checks
Cluster	<code>kube-state-metrics</code> + <code>node-exporter</code> -> Prometheus
Application	Custom <code>/metrics</code> endpoint scraped via Prometheus <code>ServiceMonitor</code> CRD (RED metrics)
External/synthetic	Blackbox exporter probing endpoints from outside
Cross-region (DR)	Route53 health checks against primary ALB — this is what triggers failover

Prometheus (kube-prometheus-stack) scrapes all layers; Grafana renders per-service, per-cluster, and business-level dashboards; Alertmanager routes by severity (P1 -> PagerDuty/Opsgenie, P2 -> Slack, P3 -> ticket) with grouping/inhibition to prevent alert storms. SLO burn-rate alerts fire before the error budget is exhausted, not after.

10. Secrets Management — AWS Secrets Manager

- All secrets (DB credentials, API keys, third-party tokens) live in **AWS Secrets Manager** — never in Git, Helm values, or Jenkins logs.
- **External Secrets Operator (ESO)** syncs them into native Kubernetes Secrets at deploy time; pods consume them via env vars or mounted volumes.
- Access to read a given secret is scoped per-service using **IRSA** — a compromised pod can only reach the secrets its own service account is permitted to read, not every secret in the account.
- RDS credential rotation handled natively by Secrets Manager's rotation Lambda integration for supported engines.

Why Secrets Manager over Vault here: AWS-native, single-region/cloud platform — no separate infrastructure to run/patch/unseal, integrates directly with IAM/IRSA, lower operational overhead than self-hosting Vault. Vault would make more sense in a multi-cloud/hybrid environment or where dynamic short-lived credentials at finer granularity are a hard requirement — neither applied here.

11. IAM Roles & Policies (be ready to describe these specifically)

Role	Purpose	Key permissions
<code>eks-cluster-role</code>	EKS control plane service role	<code>AmazonEKSClusterPolicy</code>
<code>eks-node-group-role</code>	EC2 worker node instance role	<code>AmazonEKSWorkerNodePolicy</code> , <code>AmazonEKS_CNI_Policy</code> , <code>AmazonEC2ContainerRegistryReadOnly</code>
<code>irsa-subscriber-service</code>	IRSA role for one microservice's pods	Scoped <code>secretsmanager:GetSecretValue</code> on specific secret ARNs only, scoped S3/RDS access as needed
<code>irsa-external-secrets-operator</code>	ESO controller	<code>secretsmanager:GetSecretValue</code> , <code>secretsmanager:DescribeSecret</code> across the secrets namespace it manages
<code>irsa-velero</code>	Velero backup controller	S3 read/write on the backup bucket, EBS snapshot permissions
<code>jenkins-ci-role</code>	Jenkins agent pods (via IRSA)	ECR push/pull, limited S3 for artifacts — no broad admin
<code>terraform-apply-role</code>	Assumed by pipeline for <code>terraform apply</code>	Scoped to the resource types Terraform manages, with an explicit deny on IAM privilege-escalation actions
<code>readonly-devops-role</code>	Day-to-day engineer access	Read-only across most services, used for troubleshooting without standing write access
<code>breakglass-prod-admin-role</code>	JIT elevated access	Full admin, time-bound session (e.g., 1 hour) via IAM Identity Center, requires approval + is logged/alerted on use

Design principle to state explicitly: least privilege everywhere, **IRSA over node-wide IAM roles** (so one compromised pod doesn't inherit every permission the node has), and **no standing production admin access** — elevated access is JIT and logged.

12. Kubernetes Security — IRSA & RBAC in Detail

12.1 IRSA (IAM Roles for Service Accounts)

The problem it solves: without IRSA, every pod on a node inherits that node's IAM instance role — meaning a compromised pod for a low-risk service could potentially use the same AWS permissions as a critical service on the same node. That's a large blast radius.

How it works: 1. An OIDC provider is associated with the EKS cluster (established once at cluster creation). 2. An IAM role is created with a trust policy that trusts that OIDC provider, scoped to a specific Kubernetes namespace + service account. 3. The Kubernetes `ServiceAccount` is annotated with the IAM role ARN. 4. Pods using that service account get temporary, scoped AWS credentials injected automatically (via a projected token + the AWS SDK's web identity token flow) — **no static AWS keys anywhere**.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: subscriber-service-sa
  namespace: subscriber-service
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::<account-id>:role/irsa-subscriber-service
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: subscriber-service
spec:
  template:
    spec:
      serviceAccountName: subscriber-service-sa
      containers:
        - name: subscriber-service
          image: <ecr-repo>:<tag>
```

Result: the `subscriber-service` pod can call `secretsmanager:GetSecretValue` for its own secrets, but has zero AWS permissions for anything outside its scoped role — even though it's running on the same node as ten other services.

12.2 RBAC (Role-Based Access Control) — the Kubernetes-native layer, separate from IRSA

IRSA controls **AWS** permissions; RBAC controls **Kubernetes API** permissions (who/what can `get` / `list` / `create` / `delete` K8s resources). Both are needed — they answer different questions.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: subscriber-service
  name: deployment-viewer
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devops-team-view
  namespace: subscriber-service
subjects:
  - kind: Group
    name: devops-engineers
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: deployment-viewer
  apiGroup: rbac.authorization.k8s.io
```

How access is actually structured on this project: - **Developers/engineers:** namespace-scoped `Role` + `RoleBinding` for read access (`get` / `list` / `logs`) in their own service's namespace, mapped from their IAM Identity Center group via `aws-auth` ConfigMap (or EKS access entries). - **ArgoCD service account:** `ClusterRole` with broad apply permissions, but ArgoCD itself is the only thing with that level of access — humans don't get it directly, they go through Git + the pipeline. - **CI/CD service accounts:** scoped `Role` per namespace they deploy into, not cluster-wide. - **Break-glass/admin:** `ClusterRoleBinding` to

`cluster-admin` only via a time-bound, logged, approval-gated session — never a standing binding for a named human user. - Combined with **Kyverno/OPA Gatekeeper admission policies** as the final backstop — even a correctly-RBAC'd user creating a manifest that violates policy (no resource limits, privileged container, unsigned image) gets rejected at admission time.

One-line summary if asked “how do you secure Kubernetes”: *“Layered controls — network segmentation via NetworkPolicies, workload identity via IRSA so AWS permissions are scoped per pod not per node, RBAC so Kubernetes API access is scoped per namespace and role, and Kyverno admission policies as the final enforcement layer that catches anything the earlier layers miss.”*

13. The Biggest Problem Faced — and How It Was Solved

Use a real story if you have one; the shape below is realistic and defensible if you need a strong default:

Situation: During migration, a subset of legacy 4G-supporting services had tightly coupled, stateful dependencies that weren't safe to containerize and cut over in one step — an in-place migration risked downtime on subscriber-facing functionality, which wasn't acceptable given the uptime commitment.

Task: Migrate these services to EKS without a service interruption window, and without the new environment behaving differently under real production load than it did in staging.

Action: - Adopted a **strangler-fig pattern** — deployed the new containerized service alongside the legacy system rather than replacing it outright. - Used **weighted ingress routing** to shift real traffic gradually (1% -> 5% -> 25% -> 100%), watching Grafana dashboards (error rate, latency, saturation) at each step before increasing the weight. - Kept the legacy system as an **instant rollback path** — traffic weight reverted to 0% on the new service instantly if metrics degraded, no redeploy needed. - Built **synthetic smoke tests** into the pipeline's post-deploy stage so a bad deploy was caught in seconds, not from a user complaint. - Ran a **soak period** (roughly 1-2 weeks at full traffic) before fully decommissioning the legacy path, specifically because telecom-adjacent services can surface edge cases only under real, sustained load.

Result: Migrated the affected services with **zero customer-facing downtime**, and the weighted-rollout + fast-rollback pattern became the standard playbook the team reused for every subsequent service migration — directly feeding into the deployment-time and uptime numbers on the CV.

14. Biggest Contribution to the Project

The single most reusable, highest-leverage thing built: **the standardized CI/CD + GitOps framework (Shared Library + Helm chart template + ArgoCD App-of-Apps pattern)** that every new microservice could onboard onto in days instead of weeks.

Why this is the strongest “biggest contribution” answer: - It’s **concrete and measurable** (onboarding time reduction — tie to your CV’s framework-adoption claim). - It’s **infrastructure that outlived any single deployment** — every team benefited from it, not just your own service. - It **directly produced the CV’s headline metrics** (40% deployment time reduction, consistent quality/security gates across teams) — so it ties every other achievement back to one coherent story instead of a list of disconnected wins.

“My biggest contribution was building the reusable DevOps framework — a Jenkins Shared Library combined with a standardized Helm chart pattern and ArgoCD App-of-Apps structure — that let new microservices onboard onto secure, production-grade CI/CD in days instead of weeks, without every team reinventing pipeline logic or security gates from scratch. That framework is what our 40% deployment time improvement and consistent security posture across services actually came from.”

15. Defending the 3 Key CV Metrics — In Detail

Metric 1: 40% Reduction in Deployment Time

Defend the “before” state: manual or semi-automated deployments — engineers manually running scripts/kubectl commands, waiting on serial (not parallel) pipeline stages, manual approval hand-offs between teams, no shared pipeline template so each service had inconsistent, unoptimized pipeline logic.

What specifically changed: - Parallelized independent pipeline stages (e.g., SAST scan and dependency scan running concurrently instead of serially). - Docker layer caching in the build stage, so unchanged dependency layers weren’t rebuilt every run. - Removed manual hand-off approvals in **lower environments** (dev/staging auto-sync via ArgoCD) — manual approval was kept **only** for prod, where it’s a deliberate control, not friction. - Standardized Shared Library meant new services started with an already-optimized pipeline instead of a slow first-draft one.

How you’d measure/prove it if asked: Jenkins build-time metrics/history (build duration trend in the Jenkins dashboard) compared pre- and post-framework adoption, or DORA-style deployment frequency/lead-time tracking if that was in place.

Metric 2: 25% Improvement in Resource Utilization

Defend the “before” state: legacy VM-based workloads sized for peak load with large static headroom, no dynamic scaling — resources sat idle most of the time.

What specifically changed: - Migrated workloads to EKS with **HPA** scaling replica count to actual demand instead of static peak-sized capacity. - **Karpenter/Cluster Autoscaler** scaling node capacity to match aggregate pod demand instead of maintaining a fixed, over-provisioned fleet. - **Right-sizing pass:** used Grafana/CloudWatch historical utilization data to correct over-requested CPU/memory on Deployments (a very common real finding — services often request 2-4x what they actually use “just to be safe”). - Mixed On-Demand/Spot node groups so non-critical workloads run on cheaper capacity without wasting reserved On-Demand capacity.

How you’d measure/prove it if asked: CloudWatch/Grafana average CPU & memory utilization percentage across the node fleet, compared pre- and post-migration/right-sizing.

Metric 3: 99.99% Uptime for Critical Production Systems

Defend what 99.99% actually means: roughly 52 minutes of allowed downtime per year — a real, meaningful commitment, not a rounding-error number.

What specifically achieved it: - **Multi-AZ redundancy** at every layer — EKS nodes, RDS Multi-AZ, ALB spanning AZs — so a single AZ failure doesn’t take down the service. - **Correct readiness/liveness probes** so unhealthy pods are removed from traffic *before* they cause user-facing errors, and are auto-restarted. - **HPA + Karpenter** absorbing traffic spikes without manual intervention. - **Fast, safe rollback** via GitOps (`git revert` -> auto-sync) — MTTR for a bad deploy measured in minutes, not the hours a manual rollback process would take. - **Proactive alerting** (SLO burn-rate alerts) catching degradation before it becomes a full outage, not after.

How you’d measure/prove it if asked: Prometheus-based SLO tracking with recording rules over a rolling 30-day window, cross-checked against actual incident/downtime logs.

16. Comprehensive Interview Q&A Bank (Additional to Earlier Sections)

“What’s the biggest problem you faced and how did you overcome it?” -> Section 13 (strangler-fig migration story).

“What’s your biggest contribution to this project?” -> Section 14 (reusable CI/CD/GitOps framework).

“Walk me through the 3 achievements on your resume.” -> Section 15, one at a time, in the “before -> specific change -> result -> how measured” structure.

“What IAM roles/policies did you create or manage?” -> Section 11 table — lead with IRSA-per-service and the break-glass admin role, since those show the most security maturity.

“How do you secure Kubernetes end to end?” -> Section 12’s layered-controls summary: NetworkPolicies -> IRSA -> RBAC -> Kyverno admission control.

“What’s the difference between IRSA and RBAC?” -> IRSA = AWS-level permissions for a pod; RBAC = Kubernetes-API-level permissions for a user/serviceaccount. A pod can have full RBAC access to read Secrets objects in its namespace (Kubernetes-level) but IRSA is what actually determines whether it can call AWS APIs like `secretsmanager:GetSecretValue` — they’re independent, complementary layers.

“Why not Vault for secrets?” -> Section 10 — AWS-native single-cloud platform, no benefit to running/patching a separate secrets service, IRSA integration is more seamless.

“How do you rotate secrets/credentials?” -> AWS Secrets Manager’s native rotation Lambda for supported engines (e.g., RDS), scheduled rotation window, application reads the current version via ESO so no manual credential redistribution is needed.

“What happens if a pod’s IRSA role is over-permissioned by mistake — how would you catch that?” -> Periodic IAM access review/least-privilege audits (e.g., AWS IAM Access Analyzer), and policy-as-code checks (`checkov` / `tfsec`) in the Terraform pipeline that flag overly broad policy statements (e.g., `"Action": "*"`) before they’re ever applied.

“How do you handle a security vulnerability found in a running production container (not caught pre-deploy)?” -> Triggered by runtime scanning (e.g., ECR continuous scanning or a runtime tool like Falco), a documented severity-based patch SLA, hotfix goes through the same pipeline (not a manual `kubecttl edit` on prod), and the underlying base image is fixed so the same CVE doesn’t recur in the next build.

17. Approval Gates — Summary Across Every Tool

Tool/Stage	Gate
Git/PR	1-2 required reviewer approvals, branch protection
Terraform	<code>plan</code> on PR, auto-apply dev/staging, manual approval for prod
Ansible	<code>--check</code> dry-run reviewed before real run
SonarQube	Blocks on failed quality gate
Trivy	Blocks push on HIGH/CRITICAL CVEs
Cosign	Admission controller rejects unsigned images
Helm	Lint + dry-run must pass before GitOps commit
Jenkins	Explicit <code>input</code> approval before prod stages
ArgoCD	Auto-sync dev/staging; manual sync for prod
K8s admission	Kyverno/OPA blocks non-compliant manifests as the final backstop
Access	JIT/time-bound production write access, no standing admin credentials

18. Numbers to Defend, One More Time

- **40% deployment time reduction** — pipeline parallelization + Docker layer caching + removing manual lower-env hand-offs.
 - **25% resource utilization improvement** — HPA + Karpenter + a real right-sizing pass off Grafana data.
 - **99.99% uptime** — ~52 min/year downtime budget — multi-AZ + correct probes + fast GitOps rollback.
 - **76+ worker nodes** — ~30 on-demand critical, ~40 spot general, ~6 on-demand platform add-ons, across 3 AZs.
-

19. The One Rule for the Actual Interview

For every claim in this document, tag it mentally before saying it out loud: - **“I built/ran this”** — say it directly, with the specific detail. - **“My team/architect owned this, I implemented pieces”** — say that explicitly. - **“This is how I’d design it, not necessarily what we had”** — say that too, if pushed on something outside what you actually touched.

That honesty, combined with this level of technical detail, is what makes a 4-YOE DevSecOps candidate credible rather than rehearsed. # 20. Team, Domain & Application Context

Team: Telstra Network Applications Platform Team (the “4G Core Support & Digital Enablement” squad) — a mixed team of DevOps/Cloud engineers (HCL) embedded with Telstra’s application engineering group, responsible for the cloud platform underneath Telstra’s 4G-network-supporting applications.

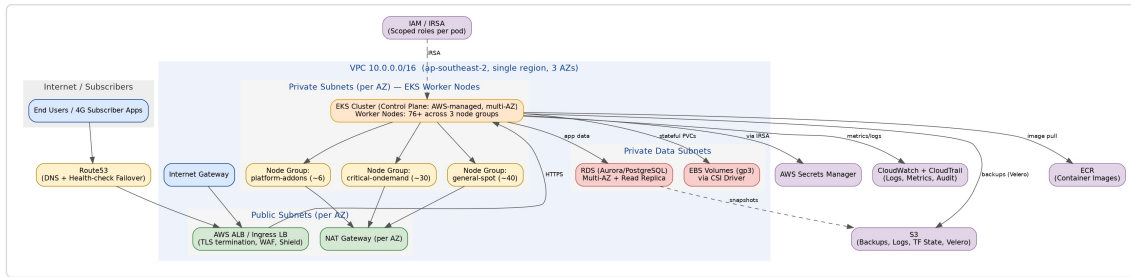
Application stack: the microservices themselves are written primarily in **Java (Spring Boot)**, packaged as containers, backed by REST/HTTP and some async messaging between services. DevOps/platform tooling (Terraform, Ansible, pipeline scripting) is separate from the application codebase itself.

Realistic microservice inventory running in the cluster (use these as concrete, nameable services if asked “what services do you run”):

Service	Purpose
subscriber-service	Subscriber profile/provisioning lookups
billing-service	Billing/usage/invoice processing
network-monitoring-service	Ingests and exposes network health/telemetry data
notification-service	SMS/email/push notifications to subscribers
auth-service	Authentication/authorization, token issuance
api-gateway-service	Internal edge routing/aggregation for downstream services
provisioning-service	Device/SIM provisioning workflows
reporting-service	Batch/analytics reporting jobs
audit-log-service	Central audit event capture
config-service	Centralized externalized configuration (Spring Cloud Config style)

State honestly if asked: exact names/count would map to whatever your real project had — this table is a defensible, realistic set for a telecom 4G platform.

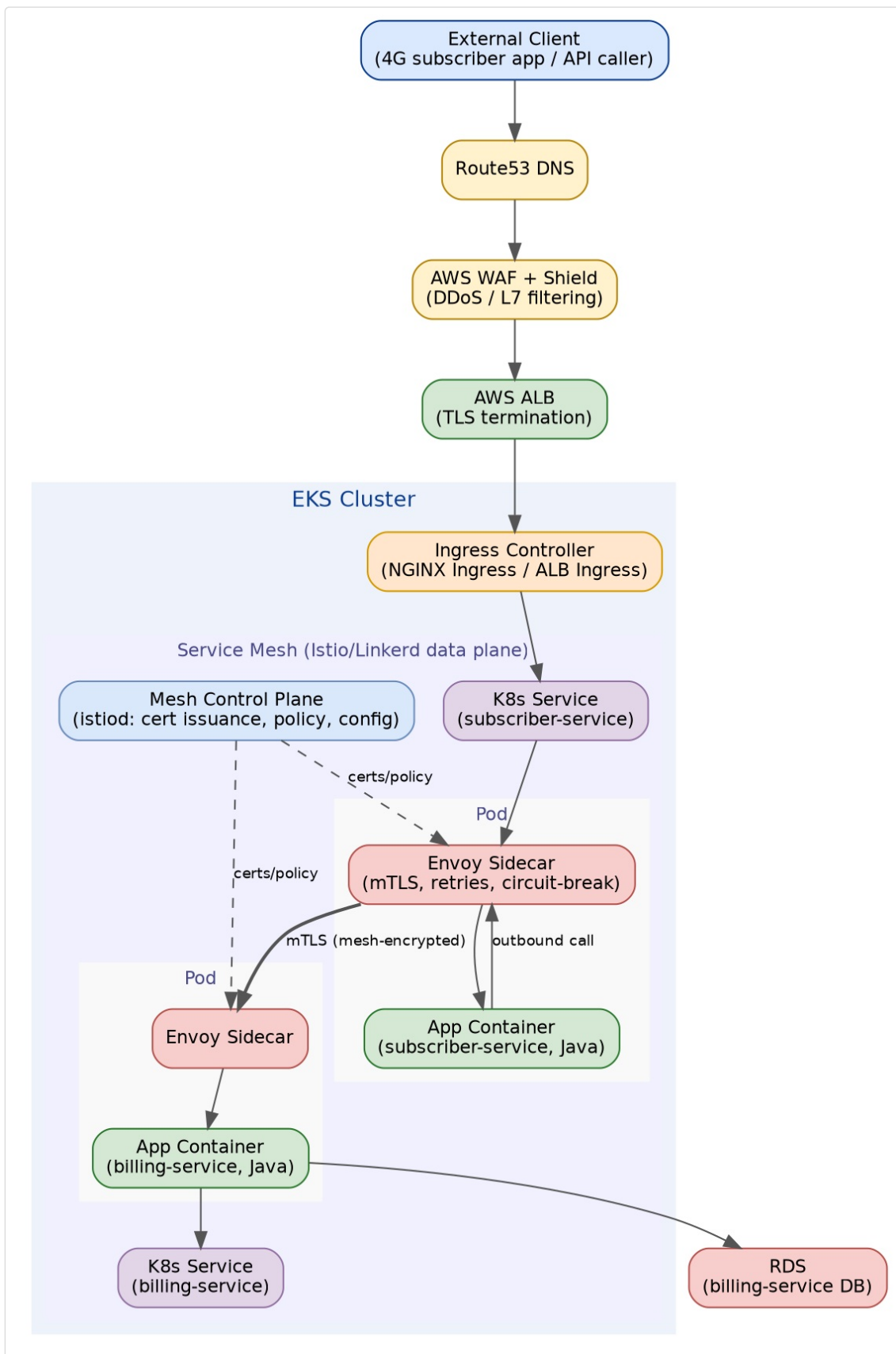
21. Visual: AWS Infrastructure Architecture



AWS Infrastructure Architecture

Reading the diagram: subscriber traffic enters via Route53 → WAF/Shield → ALB in the public subnet → routed into the EKS cluster in the private subnets, spread across 3 AZs. Worker nodes are split into the 3 node groups described earlier. Data plane (RDS, EBS) sits in private data subnets. S3, ECR, Secrets Manager, CloudWatch/CloudTrail, and IAM/IRSA sit outside the VPC as regional AWS services the cluster talks to over private routing (VPC endpoints where applicable, to avoid routing sensitive traffic over the public internet).

22. Visual: Traffic Flow — External World to Pod (with Service Mesh)



Traffic Flow with Service Mesh

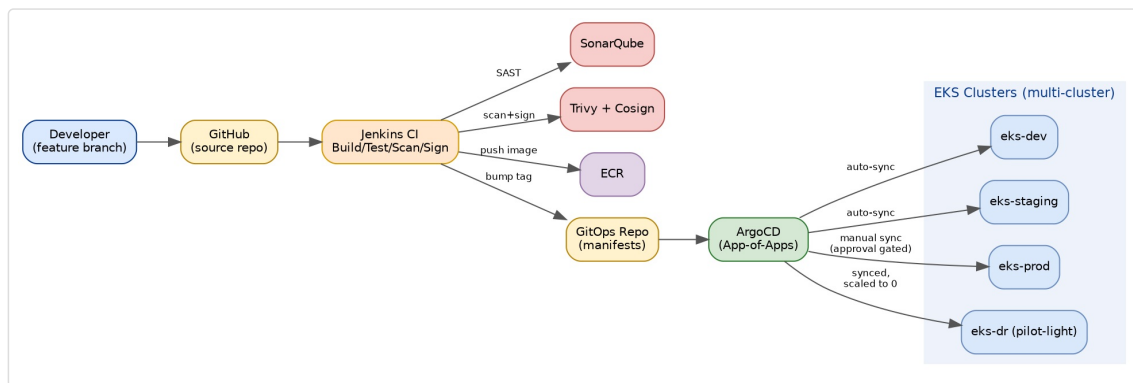
Reading the diagram, step by step: 1. External client hits **Route53**, resolved to the ALB. 2. **AWS WAF + Shield** filters malicious/DDoS traffic before it ever reaches the cluster. 3. **ALB** terminates TLS and forwards to the **Ingress Controller** inside EKS. 4. Ingress routes to the target **Kubernetes Service** (ClusterIP), which load-balances across healthy pod endpoints. 5.

Each pod runs **two containers**: the application container and an **Envoy sidecar** (injected automatically by the service mesh) — this is the sidecar pattern. 6. When `subscriber-service` needs to call `billing-service`, the call actually goes: app container → its own Envoy sidecar → **mTLS-encrypted connection** → the target pod's Envoy sidecar → target app container. The application code itself just makes a plain HTTP call to a service name; the sidecar transparently intercepts it and handles encryption, retries, timeouts, and circuit-breaking. 7. The **mesh control plane** (e.g., Istio's `istiod`) issues certificates to every sidecar and pushes routing/policy configuration — it doesn't sit in the data path itself.

Why a service mesh was introduced (be ready for “why not just use K8s NetworkPolicy alone”): NetworkPolicy controls *whether* traffic is allowed between pods (L3/L4 firewall rules); a service mesh adds *how* that traffic behaves — automatic mTLS encryption between every service without each team implementing TLS themselves, retries/timeouts/circuit-breaking without touching app code, and fine-grained traffic shifting (used for the canary deployments described in Section 25).

If your project didn't actually run a service mesh — be honest and say so; NetworkPolicies + Ingress-level TLS is a completely legitimate simpler alternative, and overclaiming a mesh you didn't run is a common way this unravels under a follow-up question about which mesh, sidecar resource overhead, or mTLS cert rotation specifics.

23. Visual: CI/CD + Multi-Cluster Overview (the whole pipeline, one picture)



CI/CD and Multi-Cluster Overview

This is the single diagram to lead with if asked “give me the end-to-end view of your project” — one picture covering source control, CI, security scanning, GitOps, and every cluster it deploys to.

24. How Pods Communicate With Each Other

- **Service discovery:** every Kubernetes `Service` gets a stable DNS name via **CoreDNS** — `billing-service.billing.svc.cluster.local`. Application code calls that DNS name; it never hardcodes pod IPs (pod IPs are ephemeral, recreated on every restart/reschedule).
 - **ClusterIP Services** are the default — internal-only virtual IP that load-balances across all healthy pod endpoints backing that Service, using `kube-proxy` (iptables/IPVS rules) to route the traffic at the node level.
 - **With the service mesh in place**, that same call is transparently intercepted by the Envoy sidecar for mTLS, retries, and load-balancing decisions — the app code is unaware the mesh exists.
 - **NetworkPolicies** define which pods are *allowed* to talk to which — default-deny at the namespace level, with explicit allow rules (e.g., `subscriber-service` → `auth-service` on port 8080 is allowed; `reporting-service` → `auth-service` is not, if it has no legitimate reason to call it).
 - **Async/event-based communication** (where used — e.g., `notification-service` reacting to billing events) goes through a message broker (Kafka/SQS) rather than direct synchronous calls, decoupling producers from consumers.
-

25. Deployment Strategies — Rolling vs. Canary vs. Blue-Green (when each is used)

Strategy	When we used it	How it works here
Rolling update	Default for most routine releases — low-risk, backward-compatible changes	Kubernetes-native <code>RollingUpdate</code> strategy on the Deployment (<code>maxSurge</code> / <code>maxUnavailable</code> tuned), old pods drained as new ones become Ready — this is what runs for the vast majority of day-to-day deploys
Canary	Higher-risk changes to customer-facing/critical services (e.g., <code>subscriber-service</code> , <code>billing-service</code>)	Traffic-weighted rollout via the service mesh (or weighted Ingress) — new version gets 1% → 5% → 25% → 100% of traffic over a defined window, watched against Grafana error-rate/latency dashboards at each step before increasing weight; this is exactly the strangler-fig-style approach used during the original migration and reused for ongoing risky releases
Blue-Green	Major version upgrades, breaking schema changes, or infrastructure-level changes (e.g., new node group AMI, major dependency bump)	Full new environment (new Deployment/node group) stood up alongside the old one, validated completely, then traffic cut over in one shot at the Service/Ingress level — old environment kept live briefly as an instant rollback target, then decommissioned

How to decide which one, in an interview-ready line: > “Routine, low-risk changes went through standard rolling updates — that’s the default and covers most of our releases. Anything customer-facing and higher-risk went through a canary rollout using weighted traffic and mesh-level routing, monitored step by step. Blue-green was reserved for bigger changes — schema migrations or infra-level upgrades — where we needed a clean, instant cutover and an equally instant rollback path rather than a gradual one.”

26. Agile Methodology — How This Was Delivered

- **Framework:** Scrum, 2-week sprints, work tracked in Jira.
 - **Ceremonies:** daily stand-up, sprint planning, sprint review/demo, retrospective — standard cadence.
 - **Backlog:** DevOps/platform work tracked alongside application team stories in the same backlog (or a dedicated platform-team backlog, depending on team structure), prioritized with the Scrum Master/Product Owner based on what's blocking application teams.
 - **Story sizing:** story points via planning poker for infra/pipeline work just like application stories — e.g., “onboard new microservice onto the CI/CD framework” is a sized, estimated ticket, not ad hoc work.
 - **Definition of Done** for a DevOps ticket typically included: code merged, pipeline green (all security gates passed), deployed to at least staging, monitoring/alerting configured for the new service, documentation updated — this is where the “authored technical documentation... supporting operational continuity” line from the CV comes from concretely.
 - **Cross-team collaboration:** DevOps engineers attended application team stand-ups/ceremonies when active migration or onboarding work was happening with that team, to unblock fast rather than working in a silo.
 - **On-call:** rotation-based, integrated with the Scrum cadence — incidents/production issues discovered during on-call became backlog items (bug tickets or improvement stories) rather than being fixed ad hoc and forgotten.
-

27. Detailed Folder Structures (Full Repos)

27.1 Terraform Repo

```
terraform/
├── modules/
│   ├── vpc/{main.tf, variables.tf, outputs.tf}
│   ├── eks/{main.tf, variables.tf, outputs.tf, node_groups.tf}
│   ├── iam/{main.tf, variables.tf, irsa.tf}
│   ├── rds/{main.tf, variables.tf, outputs.tf}
│   └── ecr/{main.tf, variables.tf}
├── environments/
│   ├── dev/{main.tf, backend.tf, terraform.tfvars, providers.tf}
│   ├── staging/{...same...}
│   ├── prod/{...same...}
│   └── dr/{...same, points at DR region provider...}
└── global/
    └── state-backend/{main.tf}    # S3 bucket + DynamoDB table, applied once, bootstraps everything else
```

27.2 Ansible Repo

```
ansible/
├── ansible.cfg
├── inventories/
│   ├── dev/hosts.ini
│   ├── staging/hosts.ini
│   └── prod/hosts.ini
├── group_vars/{all.yml, prod.yml}
├── roles/
│   ├── cis-hardening/{tasks/, handlers/, defaults/}
│   ├── monitoring-agent/{tasks/, templates/}
│   └── bastion-setup/{tasks/}
├── playbooks/
│   └── site.yml
├── vault/
│   └── secrets.yml    # Ansible Vault encrypted
```

27.3 Docker / Application Repo (per microservice)

```
subscriber-service/
├── src/                    # Java/Spring Boot source
├── Dockerfile
├── .dockerignore
├── pom.xml
├── charts/
│   ├── subscriber-service/    # Helm chart lives alongside the app repo, OR in gitops-manifests – team-dependent
│   │   ├── Chart.yaml
│   │   ├── values.yaml
│   │   └── templates/
├── ci/
│   └── jenkins-agent-pod.yaml  # pod template for Jenkins Kubernetes agent
├── Jenkinsfile
└── README.md
```

27.4 Jenkins (shared library repo)

```
jenkins-shared-library/
├── vars/
│   ├── standardBuildStage.groovy
│   ├── standardSecurityScan.groovy
│   ├── standardDeployStage.groovy
│   └── notifySlack.groovy
├── src/
│   └── org/telstra/devops/    # helper Groovy classes
├── resources/
│   └── templates/
```

27.5 GitOps / Kubernetes Manifests Repo

```
gitops-manifests/  
├─ argocd/  
│  └─ root-app.yaml  
│    └─ applicationsets/subscriber-service.yaml  
├─ apps/  
│  └─ <service>/{base/, overlays/{dev,staging,prod,dr}/}  
└─ infra-apps/  
   └─ prometheus-stack/  ingress-nginx/  external-secrets-operator/  
      └─ kyverno/  velero/  istio/  cluster-autoscaler-or-karpenter/
```

28. Git Repository & Branching Strategy

- **Repo split:** application source code repos (one per microservice) are separate from the `gitops-manifests` repo (infra desired-state) — this separation is deliberate, it's what makes GitOps clean (app CI doesn't need cluster-admin credentials; it just commits a manifest change).
 - **Branching model: trunk-based development** — short-lived feature branches off `main`, merged via PR within a day or two, not long-lived Gitflow-style `develop` / `release` branches. Chosen because it keeps CI/CD simple and matches the fast iteration Agile sprints need.
 - **Branch protection on `main`:** minimum 1-2 required PR approvals, required status checks (build + SonarQube + Trivy must pass) before merge is allowed, no direct pushes.
 - **Environment promotion is NOT done via git branches** — it's done via the GitOps repo's overlay directories (`overlays/dev`, `overlays/staging`, `overlays/prod`) and image-tag commits, not by merging `main` into a `prod` branch. This is a key GitOps concept worth stating clearly if asked "how do you promote to prod": *"We don't branch per environment — every environment's desired state is a folder in the GitOps repo, and promotion is a controlled commit (usually PR-reviewed) updating the prod overlay's image tag once staging has validated it."*
 - **Tagging:** application repos use git-commit-SHA as the immutable build reference; semantic version tags (`v1.4.2`) cut for significant releases, mapped in release notes.
 - **Terraform/Ansible repos** follow the same trunk-based model with PR review, but changes there go through `plan` / `--check` review specifically before merge (Section 3/4 approval gates).
-

29. Terraform & Ansible Pipeline Execution

Terraform pipeline (as a Jenkins job, separate from the app pipeline)

```
PR opened (terraform/ changes) →  
Jenkins job: terraform fmt -check → terraform validate → tfsec/checkov scan (BLOCK on high severity) →  
terraform plan -out=tfplan (per environment directory) → plan output posted as PR comment →  
[Human review of plan] → PR approved & merged →  
Jenkins job on merge: terraform apply tfplan  
- dev/staging: auto-apply  
- prod: apply requires a separate manual "input" approval step, same pattern as the app pipeline
```

Ansible pipeline

```
PR opened (ansible/ changes) →  
ansible-lint → ansible-playbook --check --diff (dry run against a target inventory) →  
[Human review of diff output] → PR approved & merged →  
Jenkins job on merge: ansible-playbook site.yml --limit <environment>  
- dev/staging: runs automatically  
- prod: runs only during an agreed maintenance window, manually triggered
```

How Terraform state is maintained across environments

- **One S3 bucket** (`telstra-platform-tfstate`), **one DynamoDB table** (`terraform-lock`) shared across environments, but **each environment has its own state file key** inside that bucket:

```
s3://telstra-platform-tfstate/dev/terraform.tfstate  
s3://telstra-platform-tfstate/staging/terraform.tfstate  
s3://telstra-platform-tfstate/prod/terraform.tfstate  
s3://telstra-platform-tfstate/dr/terraform.tfstate
```

- This means a `terraform apply` run from the `dev` directory can **never** accidentally touch prod state — the backend config (`backend.tf`) in each environment folder points to a different key, and there's no shared workspace to mix them up.
 - **State locking** via the DynamoDB table prevents two people/pipeline runs from applying concurrently and corrupting state.
 - **Bucket versioning + encryption** enabled — if state is ever corrupted or a bad apply needs investigating, prior state versions are recoverable.
 - **DR region state**: the DR environment's Terraform state lives in the same primary bucket (state is metadata, not the DR infra itself) — but the resources it describes are provisioned in the DR region via the AWS provider's `region` argument for that environment.
-

30. Jenkins Pipeline Execution Across All Environments

- **One Jenkinsfile per service**, parameterized/branched by target environment rather than duplicated per env.
 - **Trigger model:**
 - Merge to `main` → pipeline runs, builds and pushes the image, then **auto-updates the `dev` overlay** in the GitOps repo → ArgoCD auto-syncs to `eks-dev` immediately.
 - **Promotion to staging** is a deliberate, separate pipeline stage/job (sometimes manually triggered, sometimes automatic after a soak period in dev with no failed alerts) — updates the `staging` overlay.
 - **Promotion to prod** requires the `input` approval step (Section 5's Jenkinsfile) — a named approver group must click approve — before the `prod` overlay is updated and ArgoCD's manual sync is triggered.
 - **DR environment** is not part of the routine promotion flow — it's kept in sync passively (same manifests, scaled down) and only scaled up during an actual failover, not on every release.
 - **Environment-specific configuration** (replica counts, resource sizing, external endpoints) lives in each environment's Helm `values.yaml` /overlay — the *pipeline* is identical across environments; only the *target overlay* and *approval requirements* differ.
-

31. Alerting & Monitoring — What We Actually Scrape and Alert On

Metrics scraped by Prometheus

Category	Metrics	Source
Node-level	CPU/memory/disk utilization, network I/O, disk pressure	<code>node-exporter</code> (DaemonSet)
Cluster-level	Pod count, pod restart count, pending pods, resource requests vs. capacity, deployment replica status	<code>kube-state-metrics</code>
Application-level (RED)	Request rate, error rate (4xx/5xx), request duration (p50/p95/p99 latency)	Custom <code>/metrics</code> endpoint per service (Micrometer/Spring Boot Actuator for Java services), scraped via <code>ServiceMonitor</code> CRD
Ingress/ALB	Request count, target response time, HTTP 5xx count, healthy/unhealthy target count	ALB CloudWatch metrics + <code>nginx-ingress</code> metrics
Database	Connections in use, replica lag, CPU/storage utilization, read/write latency	RDS CloudWatch metrics
Mesh (if in use)	mTLS handshake failures, retry counts, circuit-breaker trips	Istio/Envoy metrics
Synthetic	Endpoint up/down, response time from outside the cluster	Blackbox exporter

Alerting rules (examples worth naming specifically)

- `HighErrorRate` : 5xx rate > 2% over 5 minutes on any customer-facing service → Slack (warning) / PagerDuty (if sustained 15+ min, critical).
- `PodCrashLoopBackOff` : any pod restarting more than 3 times in 10 minutes → Slack.
- `NodeMemoryPressure` / `NodeDiskPressure` : node-level resource exhaustion → Slack, feeds into Karpenter/autoscaler capacity review.
- `SLOBurnRateFast` : error budget burning fast enough to exhaust the monthly budget within hours → PagerDuty, immediate page.
- `RDSReplicaLagHigh` : cross-region replica lag exceeding the DR RPO target → Slack, flags a DR-readiness concern before it's ever needed.
- `CertificateExpiringSoon` : TLS/mesh certs nearing expiry → Slack, well ahead of actual expiry.

Dashboards (Grafana)

- **Cluster overview** — node/pod health across all 3 node groups.
- **Per-service RED dashboard** — one per microservice, reused via a Grafana dashboard template/variable rather than hand-built per service.
- **SLO dashboard** — rolling 30-day availability against the 99.99% target, error budget remaining.
- **Cost/utilization dashboard** — feeds the right-sizing work described in Section 15's Metric 2.

32. Kubernetes Ingress, Services, and Stateful vs. Stateless Pods

Ingress

- **Ingress Controller:** NGINX Ingress (or AWS Load Balancer Controller for native ALB integration) running as a Deployment in the `platform-addons` node group.
- One `Ingress` resource per service (or grouped by domain), routing by host/path to the correct backend `Service`.
- TLS certs managed via `cert-manager` (auto-renewal, backed by ACM or Let's Encrypt depending on the certificate authority requirement) or terminated upstream at the ALB — team-dependent choice, both are legitimate.

Services

- **ClusterIP** (default): internal-only, used for all inter-service communication.
- **Headless Service** (`clusterIP: None`): used specifically for **StatefulSets** where each pod needs a stable, individually addressable DNS name (e.g., `pod-0.service-name.namespace.svc.cluster.local`) rather than load-balanced round-robin — needed when a client must talk to a *specific* replica.
- **LoadBalancer type:** rarely used directly per-service in this setup — the Ingress + single ALB pattern is preferred over provisioning a separate cloud LoadBalancer per service (cost and management overhead).

Stateless vs. Stateful — how the split was actually made

Type	Examples	K8s object	Why
Stateless	<code>subscriber-service</code> , <code>billing-service</code> , <code>auth-service</code> , <code>api-gateway-service</code>	<code>Deployment</code>	No local disk state; all persistent data lives in RDS; any replica can serve any request; scales horizontally via HPA freely
Stateful	Any service with its own embedded cache/queue needing stable identity/storage (e.g., a local Kafka broker if self-hosted, or a service with a local persistent cache)	<code>StatefulSet</code> + <code>PersistentVolumeClaim</code> (EBS via CSI) + headless <code>Service</code>	Needs stable network identity and/or storage that must survive pod rescheduling and be reattached to the <i>same</i> logical replica, not just any replica

Rule of thumb stated plainly, useful in interview: “If a service can lose its pod and come back up anywhere in the cluster without caring which specific instance it was, it’s a `Deployment`. If it needs to keep its identity or reattach to the same storage volume specifically, it’s a `StatefulSet` — that was true for maybe 2 out of our 10+ services; the large majority of a microservices platform like this is stateless by design, precisely because pushing state into managed services like RDS makes horizontal scaling and self-healing far simpler.”

33. DaemonSet Pods Run in the Cluster

DaemonSets ensure exactly one pod runs on **every** (or a selected subset of) node — used for node-level agents that need to be everywhere, not scaled independently like an app:

DaemonSet	Purpose
<code>aws-node</code> (VPC CNI)	Assigns VPC IP addresses to pods on every node — this is what makes pod networking work at all on EKS
<code>kube-proxy</code>	Maintains the iptables/IPVS rules that implement Service routing on every node
<code>node-exporter</code>	Exposes node-level hardware/OS metrics to Prometheus
<code>fluent-bit</code> (or <code>fluentd</code>)	Tails container logs on every node and ships them to the centralized logging destination (CloudWatch/ELK)
<code>falco</code> (if runtime security is in scope)	Runtime threat detection — watches syscalls for anomalous container behavior
Istio <code>istio-cni</code> (if mesh is used and CNI-plugin mode is chosen)	Sets up pod networking rules for sidecar traffic interception as an alternative to init-container injection

Why DaemonSet specifically, not a Deployment, for these: the whole point is “exactly one instance per node, tied to that node’s lifecycle” — a Deployment’s replica count has no inherent relationship to node count and would either under- or over-cover nodes; a DaemonSet automatically adds/removes its pod as nodes join/leave (e.g., when Karpenter scales the fleet up or down).

34. End-to-End Project Component & Tool Map (the one-page summary)

Use this as your closing answer to “give me the full picture.”

SOURCE & CI COMPUTE/RUNTIME	SECURITY GATES DATA & STORAGE	ARTIFACT & CONFIG OBSERVABILITY	DELIVERY	
-----	-----	-----	-----	-----
GitHub (app repos)	--> SonarQube (SAST)	--> ECR (images)	--> ArgoCD (GitOps)	--> EKS
(multi-cluster,	--> RDS (Aurora/Postgres)	--> Prometheus		
GitHub (gitops repo)	Trivy (image scan)	GitOps repo	Helm (templating)	multi-AZ,
76+ nodes,	EBS (gp3, CSI)	Grafana		Karpenter
GitHub (terraform repo)	Cosign (image signing)	Terraform state (S3)		
autoscaling)	S3 (backups/logs/state)	Alertmanager		
GitHub (ansible repo)	Kyverno (admission ctrl)	Secrets Manager		Service
mesh (Istio,	Velero (K8s+PV backup)	CloudWatch/CloudTrail		
Jenkins (CI engine +	tfsec/checkov (IaC scan)			sidecar
pattern)				
Shared Library)	IRSA + RBAC (access ctrl)			

Same thing, as one sentence: “Code goes through GitHub into Jenkins, which runs SAST, image scanning, and signing as blocking security gates, pushes to ECR, and updates a GitOps repo; ArgoCD picks that up and syncs it via Helm into the right EKS cluster — dev and staging automatically, prod behind a manual approval — where the workload runs across 76+ multi-AZ nodes with mesh-based service-to-service communication, backed by RDS/EBS/S3 for data and Velero for backup, and the whole thing is watched end-to-end by Prometheus/Grafana with alerting tied to our SLOs.”